AIDA vs Karpathy-style structured markdown

AIDA positioning · best-effort comparison — live source: github.com/joemooney/aida

rendered 2026-07-10

Last updated: 2026-07-09

The TL;DR: **structured markdown queryable by Claude is the floor.** It's a real, working baseline for AI-assisted project context — and for many projects it's enough. AIDA's pitch is that **once your project crosses a complexity threshold, you start wanting the things markdown can't give you: stable IDs, a relationship graph, records you query instead of grep, and an enforcement loop.** This page is about where that threshold is.

What "Karpathy-style markdown" means here

Andrej Karpathy's working pattern (popularized via X / public talks 2024–2025):

- Treat the project repo's markdown files (`CLAUDE.md`, `OVERVIEW.md`, `docs/*.md`) as a queryable index.
- Whenever an agent needs context, point it at the relevant file. Whenever a decision is made, append it to the relevant file.
- The "database" is the file tree; the "query language" is grep + the agent's reading comprehension.

This is **a genuinely good baseline.** It costs nothing to set up, has zero infrastructure, and survives any tooling change. For a solo developer on a small project, it might be all that's ever needed.

CLAUDE.md and OVERVIEW.md in *this* repo are themselves examples of the pattern.

Where structured markdown alone holds up

- **Project size**: one developer, one repo, a few hundred decisions over the project's lifetime.
- **Context shape**: most of what an agent needs to know fits in a few documents. *"Read CLAUDE.md, then answer the question"* works.

- **Reference style**: cross-references can be informal — "the auth flow" is a unique enough phrase to grep.
- **Audit needs**: low. Nobody is going to ask *"prove that this code change traces to that requirement."*
- **Tool integration**: minimal. The agent's prompt is the only consumer of the markdown.

If all five of these match, **don't deploy AIDA.** The friction-to-value ratio doesn't work out.

Where structured markdown alone starts to crack

Symptom	What markdown can't do about it
"We discussed this last week — what did we decide?"	Grep finds the discussion but not the resolution; nothing forces a decision to be recorded as one
"What's the status of feature X?"	Status is a property of work, not a section of a doc; markdown can hold it but doesn't enforce it
"What depends on this code path?"	No graph — the relationship has to be in someone's head
"Are any acceptance criteria not met?"	Acceptance criteria as bullet lists can't be machine-walked against a diff
"Rename FR-0042"	Without stable IDs, every cross-reference is a string match the agent might or might not catch
"Show me everything tagged 'auth'"	Filterable views need a query layer — `find . -name '*.md' | xargs grep auth` returns text, not records
"Two agents touched this in parallel and disagreed"	No conflict-detection layer; whoever pushed last wins silently

These symptoms are the **entry conditions for AIDA.** The threshold isn't "X requirements" or "Y developers" — it's the first time you wish your markdown was queryable as records, not text.

What AIDA adds on top

AIDA's defensible niche statement (from OVERVIEW.md and CLAUDE.md):

> The agent-collaboration layer: **stable spec IDs, typed relationships, code-to-spec trace comments, and an MCP server that exposes the**

requirement graph to coding agents. Karpathy-style "structured markdown queryable by Claude" is the floor; AIDA adds the relationship graph + identifier stability + enforcement loop.

Concretely, the four primitives AIDA adds that pure markdown can't reach:

1. **Stable spec IDs.** `FR-1`, `STORY-104`, `EPIC-24` — these resolve to the same UUID forever, so trace comments in code and references in commit messages don't rot. Markdown headings can be renamed at any time; markdown can't promise stability.
2. **Typed relationships.** `parent` / `child` / `verifies` / `references` edges between specs render as a graph. The agent walks the graph instead of grepping for nearby paragraphs.
3. **Queryable as records, not text.** A token-efficient CLI is the primary agent surface (`aida list/search/show`, machine output via `AIDA_AGENT_OUTPUT`), with `aida mcp-serve` exposing the same graph as typed MCP tools for MCP-native clients. Either way the graph is queried as a database, where markdown is only ever loaded as plain context.
4. **Code-to-spec trace comments.** `// trace:FR-1-042 | ai:claude` in code, walked by `aida trace`, completes the round-trip. The graph isn't an aspirational sidecar — it's enforced against the code.

The enforcement loop is the load-bearing piece. AIDA isn't just "markdown with a schema" — it's **markdown with a check that the schema agrees with the code** — a check that is real, enforced code (e.g. the pre-work authority gate and the pr-ship self-merge guard), not a convention.

Composition: use both, intentionally

The realistic deployment is not *"replace markdown with AIDA."* It's **"keep your CLAUDE.md / OVERVIEW.md / WHY-AIDA.md narrative; add AIDA underneath for the parts markdown can't carry."**

In this repo:

- CLAUDE.md — orientation prose for new sessions. Not in the graph.
- OVERVIEW.md — vision document. Not in the graph.
- `aida-store` orphan branch — the graph. Source of truth for *which specs exist, how they relate, what status they're in*.
- `docs/positioning/` (this directory) — narrative built from `aida doc` entries; renders the graph's "why" into prose.
- `aida docs build` — projects the graph into a layered docs tree (constitution / vision / decisions / quality / glossary) for human reading.

The boundary is: **markdown for prose humans read top-to-bottom; the graph for records agents query selectively.** Neither replaces the other.

When to deploy AIDA

A rough decision tree:

1. *"Can I name every meaningful decision in my project from memory?"* Yes → markdown is fine.
2. *"Do I need agents to consult project context across sessions?"* No → markdown is fine.
3. *"Will I want to ask 'show me everything that depends on X' in a year?"* If yes → AIDA is earning its keep.
4. *"Does my project span more than one repo or developer?"* If yes → AIDA's graph + stable IDs become significantly more valuable than markdown.
5. *"Am I shipping AI-assisted code I can't yet trace back to intent?"* If yes → AIDA's trace comments + enforcement loop is the answer.

You'll know you've hit the threshold when grep stops being enough.

See also

- CLAUDE.md — orientation prose this repo uses alongside AIDA.
- OVERVIEW.md — vision document, also written as markdown not graph records.
- vs-saas-pm.md — when AIDA's graph layer competes with Linear/Jira instead of with markdown.